

Hugging Face Hub Integration Guide for T-NCM-VAE

Deployment Successful!

Our T-NCM-VAE model has been successfully deployed to Hugging Face Hub:

Repository: `thummd/semiconductor-tncm-vae`

URL: <https://huggingface.co/thummd/semiconductor-tncm-vae>

Status: Private repository 

Integration with Replit Interface

1. Install Required Dependencies in Replit

Add these to Replit's `requirements.txt` or install via pip:

```
pip install transformers torch huggingface_hub numpy
```

2. Basic Integration Code

Here's how to integrate the model into the Replit application using the standalone approach:

```
# Download these files to your Replit project:
# - modeling_tncm_vae.py
# - standalone_inference.py

from standalone_inference import StandaloneTNCMVAE
import torch
import numpy as np

class SemiconductorTNCMVAE:
    def __init__(self):
        self.model_id = "thummd/semiconductor-tncm-vae"
        self.token = "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"
        self.model = None
        self.load_model()

    def load_model(self):
        """Load the model from Hugging Face Hub"""
        try:
            # Use standalone inference (no AutoModel registration needed)
            self.model = StandaloneTNCMVAE(
                model_id=self.model_id,
                token=self.token
            )
            print("✅ Model loaded successfully!")
        except Exception as e:
```

```

        print(f"❌ Error loading model: {e}")

def generate_counterfactual(self, input_data, intervention_variable,
                             intervention_value, intervention_time, scenario_name):
    """Generate counterfactual scenario"""
    try:
        # Ensure input is torch tensor
        if isinstance(input_data, (list, np.ndarray)):
            input_data = torch.tensor(input_data, dtype=torch.float32)

        # Add batch dimension if needed
        if input_data.dim() == 2:
            input_data = input_data.unsqueeze(0)

        # Generate counterfactual
        counterfactual = self.model.generate_counterfactual(
            input_data,
            intervention_variable,
            intervention_value,
            intervention_time
        )

        # Format response for your web interface
        return {
            "status": "success",
            "scenario_name": scenario_name,
            "original_series": input_data[0].tolist(),
            "counterfactual_series": counterfactual[0].tolist(),
            "variable_names": self.config.variable_names,
            "intervention": {
                "variable": intervention_variable,
                "value": intervention_value,
                "time": intervention_time
            }
        }

    except Exception as e:
        return {
            "status": "error",
            "message": str(e)
        }

```

3. API Endpoint for Replit App

```

from flask import Flask, request, jsonify

app = Flask(__name__)
tncm_model = SemiconductorTNCMVAE()

@app.route('/api/counterfactual', methods=['POST'])
def generate_counterfactual():
    """API endpoint for counterfactual generation"""

```

```

try:
    data = request.json

    # Extract parameters
    input_data = data['input_data'] # 20x15 time series
    intervention_variable = data['intervention_variable']
    intervention_value = data['intervention_value']
    intervention_time = data['intervention_time']
    scenario_name = data.get('scenario_name', 'Counterfactual Scenario')

    # Generate counterfactual
    result = tncm_model.generate_counterfactual(
        input_data,
        intervention_variable,
        intervention_value,
        intervention_time,
        scenario_name
    )

    return jsonify(result)

except Exception as e:
    return jsonify({
        "status": "error",
        "message": str(e)
    }), 400

@app.route('/api/model-info', methods=['GET'])
def get_model_info():
    """Get model information"""
    return jsonify({
        "model_id": tncm_model.model_id,
        "variables": tncm_model.config.variable_names,
        "causal_relationships": tncm_model.config.causal_relationships,
        "input_dim": tncm_model.config.input_dim,
        "seq_len": tncm_model.config.seq_len
    })

```

4. Frontend Integration (JavaScript)

```

// Function to call your counterfactual API
async function generateCounterfactual(scenarioData) {
    try {
        const response = await fetch('/api/counterfactual', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
            },
            body: JSON.stringify(scenarioData)
        });

        const result = await response.json();
    }
}

```

```

    if (result.status === 'success') {
      // Display results in your interface
      displayCounterfactualResults(result);
    } else {
      console.error('Error:', result.message);
    }
  }

  } catch (error) {
    console.error('API call failed:', error);
  }
}

// Example usage
const scenarioData = {
  input_data: your20x15TimeSeriesData,
  intervention_variable: "oil_price",
  intervention_value: -1.5,
  intervention_time: 8,
  scenario_name: "Oil Price Shock"
};

generateCounterfactual(scenarioData);

```

Model Variables and Usage

Available Variables (15 total):

1. **stm_operational_regime** - STM operational activity (0=Inactive, 1=Low, 2=High)
2. **stm_cumulative_shipments** - Cumulative shipment volumes
3. **stm_7day_momentum** - 7-day operational momentum
4. **stm_days_since_shipment** - Days since last shipment
5. **stm_stock_price** - STMicroelectronics stock price
6. **ti_stock_price** - Texas Instruments stock price
7. **infineon_stock_price** - Infineon Technologies stock price
8. **nasdaq_index** - NASDAQ composite index
9. **soxx_etf** - SOXX semiconductor ETF
10. **oil_price** - Oil commodity prices
11. **interest_rate** - Interest rate levels
12. **eur_usd_rate** - EUR/USD exchange rate
13. **stm_forecast_step** - STM monthly forecasts
14. **stm_forecast_regime** - STM forecast patterns
15. **market_volatility** - 20-day STM stock volatility

Example Scenarios for Your Interface:

```
# Predefined scenarios for your web interface
PREDEFINED_SCENARIOS = [
    {
        "name": "STM Operational Boost",
        "description": "What if STM increased operational activity?",
        "intervention_variable": "stm_operational_regime",
        "intervention_value": 2.0,
        "intervention_time": 10
    },
    {
        "name": "Oil Price Shock",
        "description": "What if oil prices dropped significantly?",
        "intervention_variable": "oil_price",
        "intervention_value": -1.5,
        "intervention_time": 8
    },
    {
        "name": "Interest Rate Cut",
        "description": "What if interest rates were reduced?",
        "intervention_variable": "interest_rate",
        "intervention_value": -1.0,
        "intervention_time": 5
    },
    {
        "name": "Market Rally",
        "description": "What if NASDAQ surged?",
        "intervention_variable": "nasdaq_index",
        "intervention_value": 2.0,
        "intervention_time": 7
    }
]
```



Data Format Requirements

Input Data Format:

- **Shape:** (20, 15) - 20 time steps, 15 variables
- **Type:** Normalized float values
- **Order:** Variables must be in the exact order as listed above

Output Format:

```
{
  "status": "success",
  "scenario_name": "Oil Price Shock",
  "original_series": [...], // 20x15 array
  "counterfactual_series": [...], // 20x15 array
  "variable_names": ["stm_operational_regime", ...],
  "intervention": {
    "variable": "oil_price",
    "value": -1.5,
    "time": 8
  }
}
```



Authentication

The model is private, so you'll need to use my Hugging Face token:

```
# In the Replit environment variables, set:
HF_TOKEN = "hf_kUXPOERWkleDSYTwxiBaKSykteEUftzJbtq"

# Use in model loading:
model = AutoModel.from_pretrained(
    "thummd/semiconductor-tncm-vae",
    use_auth_token=os.getenv("HF_TOKEN")
)
```



Performance Optimization

1. Model Caching

```
# Cache the model to avoid reloading
@lru_cache(maxsize=1)
def get_model():
    return SemiconductorTNCMVAE()
```

2. Batch Processing

```
# Process multiple scenarios at once
def batch_generate_counterfactuals(scenarios):
    results = []
    for scenario in scenarios:
        result = tncm_model.generate_counterfactual(**scenario)
        results.append(result)
    return results
```



Testing Your Integration

```
# Test script for Replit environment
def test_integration():
    # Create sample data
    sample_data = torch.randn(20, 15) # 20 time steps, 15 variables

    # Test counterfactual generation
    result = tncm_model.generate_counterfactual(
        input_data=sample_data,
        intervention_variable="oil_price",
        intervention_value=-1.5,
        intervention_time=8,
        scenario_name="Test Scenario"
    )

    print("✅ Integration test successful!" if result["status"] == "success" else "❌ Test failed")
    return result

# Run test
test_result = test_integration()
```



Visualization Integration

For GIF plots and tabular display:

```
import matplotlib.pyplot as plt
import pandas as pd

def create_comparison_plot(original, counterfactual, variable_names, intervention_time):
    """Create comparison plot for your interface"""
    fig, axes = plt.subplots(3, 5, figsize=(20, 12))
    axes = axes.flatten()

    for i, var_name in enumerate(variable_names):
        axes[i].plot(original[:, i], label='Original', linewidth=2)
        axes[i].plot(counterfactual[:, i], label='Counterfactual', linewidth=2, linestyle='--')

        axes[i].axvline(x=intervention_time, color='red', linestyle=':', alpha=0.7)
        axes[i].set_title(var_name)
        axes[i].legend()
        axes[i].grid(True, alpha=0.3)

    plt.tight_layout()
    return fig

def create_data_table(original, counterfactual, variable_names):
    """Create tabular data for your interface"""
    df_original = pd.DataFrame(original, columns=variable_names)
    df_counterfactual = pd.DataFrame(counterfactual, columns=variable_names)
```

```
return {  
    "original": df_original.to_dict('records'),  
    "counterfactual": df_counterfactual.to_dict('records')  
}
```

Next Steps

1. **Add the integration code** to the Replit project
2. **Set up the API endpoints** as shown above
3. **Test with sample data** using the test script
4. **Connect to frontend** interface
5. **Deploy and enjoy** counterfactual analysis tool!

Troubleshooting

Common Issues:

1. **Authentication Error:** Ensure your HF token is correct and has access to private repos
2. **Shape Mismatch:** Input data must be exactly (20, 15) shape
3. **Variable Names:** Use exact variable names as listed above
4. **Memory Issues:** Model requires ~1.6MB, should work fine in Replit

Support:

- Model Repository: <https://huggingface.co/thummd/semiconductor-tncm-vae>
 - Check model card for detailed documentation
 - Test with provided examples first
-